

Project 2 Photostitch Panorama

Introduction

Photostitch is the process of merging multiple photographs together to create a single high resolution or panoramic image.[1] As we now call Pano or Panorama, a similar technique is applied here to achieve such result to make multiple image of similar viewpoint together into one.

Problem Statement:

To implement a pipeline for panorama stitching, combining individual photos with overlapping fields of view to produce a panorama image.

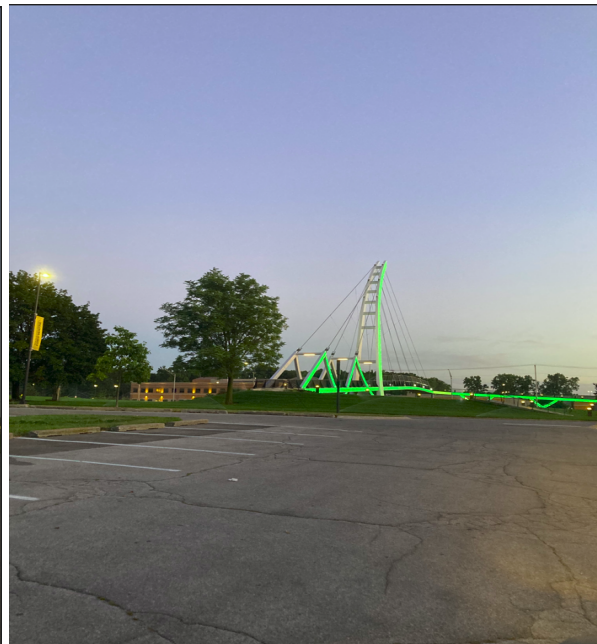
Discussion:

This report presents the assigned discussion, having much acquired knowledge the images were worked on with understanding the similar matching-points found for each similar structure found from one image to the other.

In terms of stitching of the image, it has been considered by locating keypoints and descriptors as the main parameters, some greyscale configuration was done to the images for better matching and detection. The ORB algorithm was introduced to bring up a better alternative to SIFT and SURF. By using the algorithm the parameters were designated with range for satisfying merging or stitching the images.[2] It has more enhancement through assigning a sorting distance to track the points and get the points aligned. Again, these points are matched in 2 images each time, that helps synchronizing the images respectively. As these are merged respectively, all the images, where finally these were put into a perfect panorama with an appropriate blend through merge function. The ultimate result was then found. The work was experimented mainly on the given images:



(a)



(b)

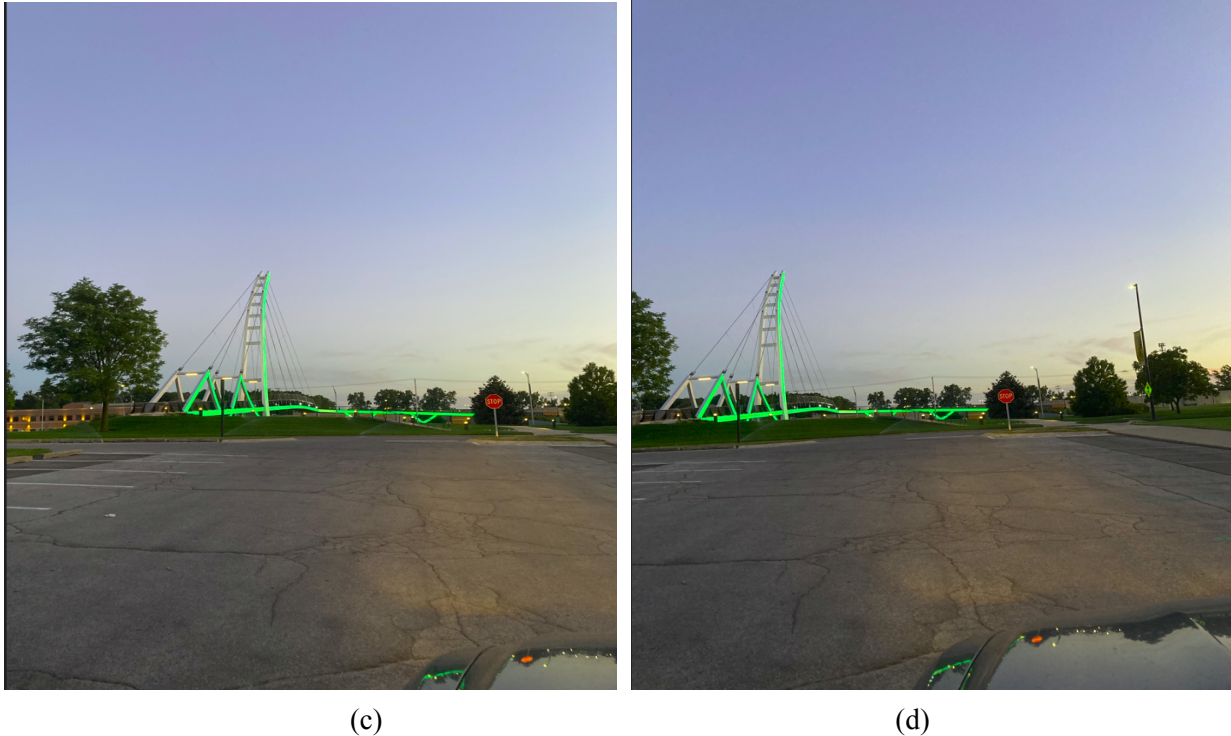


Figure 1: The images assigned for image processing; (a) image1.jpg, (b) image2.jpg, (c) image3.jpg, (d) image4.jpg

Design and Implementation

The design was implied based on the image orientation and some experimentation concept in head for being able to map the image proportionately. The stated guidelines:

- Key point detection
- Local invariant descriptor extraction
- Corresponding key point matching
- Outlier detection and homography matrix estimation using matched feature vectors
- Image warping and blending

All the above criteria were optimally aimed and worked for implementation.

Platform: In Google Colab

Mode of use and support: Python, OpenCV concepts, RANSAC, ORB, Direct and Blending Mechanisms

Steps to design and implementation:

- (Imports) It imports the required libraries; “OpenCV (cv2)”, “sys” and ‘NumPy (np)’.

- (Image Load and Conversion) The 4 images are loaded and then converted to a Grayscale with the function 'cv2.cvtColor'. Features can be found and matched using grayscale image, as an image provided below;



Figure 2: “image1.jpg” in grayscale.

- (Feature Detection and Descriptor Extraction) With `cv2.ORB_create(nfeatures=2000)`, the code generates an ORB (Oriented FAST and Rotated BRIEF) feature detector and descriptor extractor. The ORB technique is widely used to identify descriptors and focal points in photographs. Again, Keypoints and descriptors for every grayscale image are found using the `detectAndCompute` function. Matching will be done using these descriptors and keypoints. [2]

- (Key Point Visualization) The discovered keypoints are drawn on the original color images by the code using `cv2.drawKeypoints`. An image showing how the point will look is given below;



Figure 3: The Keypoints detection in “image1.jpg”

- (Feature Matching) Applying `cv2.BFMatcher (cv2.NORM_HAMMING, crossCheck=True)` yields a Brute-Force Matcher (BFMatcher). Matching descriptors will be the use of it. Every pair of neighboring images—images 1 to 2, image 2 to 3, and image 3 to image 4—has its descriptor matched. Using sorted, matches are arranged according to distance. The first N matches for every pair are kept in the code.
- (Homography Calculation) The RANSAC method is used to compute a homography matrix for each pair of adjacent images using the code `cv2.findHomography`. The alignment of the images is represented by this matrix, which shows how the images were transformed.

From here onward, different processes were experimented with to achieve the right result. Following are the different turns taken after Homography was brought in to it;

Model 1 (Direct Library function for Gradient Blending)

- (Warp Images) The computed homography matrix is used to warp the current image onto the final image. The photos are correctly aligned after this stage.
- (Gradient Domain Blending) Building a mask and implementing the `cv2.seamlessClone` function with the `cv2.NORMAL_CLONE` mode to smoothly integrate the warped image into the output image.

- (Iterative Stitching) Iterating through the images, gradually blending, warping, and calculating the homography between each one to create the final image, each image in the panorama is added using this method.
- (Panoramic Image Display and Save) cv2_imshow is used to display the final panoramic image and the code employs cv2.imwrite to save the stitched panoramic image to a file with the name "panorama_GradLib.jpg".

Model 2 (Direct Image Concatenation)

- (Panorama Dimensions Calculation) Based on the dimensions of the individual images, the code determines the dimensions of the stitched panoramic image.
- (Making Panorama Canvas) The dimensions are calculated and used to create an empty black canvas (panorama).
- (Image Warping and Stitching) After being warped, the images are positioned on the panorama canvas.
 - The first image (image1) is positioned exactly where it was originally.
 - Utilizing the homography matrix (M12), the second image (image2) is warped and positioned next to the first image.
 - The third image (image3) is positioned next to the second image after being warped using the homography matrices $M12 \cdot M23$ and image3.
 - The third image is placed next to the fourth image, which is also warped.
- (Panoramic Image Display and Save) cv2_imshow is used to display the final panoramic image and the code employs cv2.imwrite to save the stitched panoramic image to a file with the name "panoramic_image.jpg".

Model 3 (Gradient Blending Breakdown Technique) [3]

- (Warp Images) Making the images fit together, warp them. Create a panorama for each set of two adjacent images by warping the first image with homography and stitching it with the second image.
- (Calculate Gradients) Making use of Sobel filters, figure out the gradient for the final stitched panorama in the x and y directions. To get ready for gradient blending, this was done.
- (Calculate Laplacian of Gradients) For the gradient images, the Laplacian in the x and y directions was determined.
- (Construct Poisson Equation Matrix) The gradient blending Poisson equation can be solved by combining the Laplacian of gradient values and creating a sparse matrix.
- (Solve Poisson Equation for Each Color Channel) Solving the Poisson equation for each of the three color channels (R, G, and B) using the formerly created matrix. Each color channel's gradients are combined in this step.
- (Panoramic Image Display and Save) cv2_imshow is used to display the final panoramic image and the code employs cv2.imwrite to save the stitched panoramic image to a file with the name "panorama_gradient.jpg".

Model 4 (Alpha Blending Technique)

- (Warp Images) Using the determined homography matrix H_{12} , warp image 1 to image 2. To make a bigger panorama canvas, combine them horizontally. Again, using the computed homography matrix H_{34} , warp picture 3 to image 4. Then turning them into a second panorama canvas by combining them horizontally.
- (Framework for Panorama) To build the panorama that was created in the stage before, warp image 2 using the homography matrix H_{23} . Combine them in a horizontal manner to create the final panoramic canvas.
- (Alpha Blending) Selecting the blending's alpha value (0.5 in this example). Alpha blending was done in this order:
 - Combine images 1 and 2 using the given alpha.
 - Utilizing the same alpha, blend the outcome with image 3.
 - Lastly, build the fully stitched panorama by blending the merged image with image 4.
- (Panoramic Image Display and Save) `cv2_imshow` is used to display the final panoramic image and the code employs `cv2.imwrite` to save the stitched panoramic image to a file with the name "panorama_alpha.jpg".

With the final steps completed, the image was opened to check. It was to be cautious that all the procedures had an image print option to have a saved file for better speculation while comparing the methods for each model.

The images were illustrated as a good practice for image stitch model of the 4 given images, as a single Panorama.

Results and Performance Evaluation

The results were mainly an output of the images merged into a panoramic image showing all the 4 images together. But within this, some comparisons were conducted to conclude some ultimate mechanism to illustrate as the most optimized ones to give a solution to creating a Panorama.

The code adjusts the input image with the help of homography and RANSAC, then uses OpenCV while implementing the direct stitching and blending mechanisms. All the procedures were accordingly maintained as stated in the above “Design and Implementation.”. The comparison for each mechanism's output were presented below to define which procedure suited the most and gave a good output;

Model 1 (Direct Library function for Gradient Blending)

For this Model the main Library function for Gradient Blending, which then with Poisson Equation were calculated as per the built-in function mechanism. But the end results didn't print-out the whole image rather illuminated the first image causing not apparent stitch at the very end. (Shown as follows);

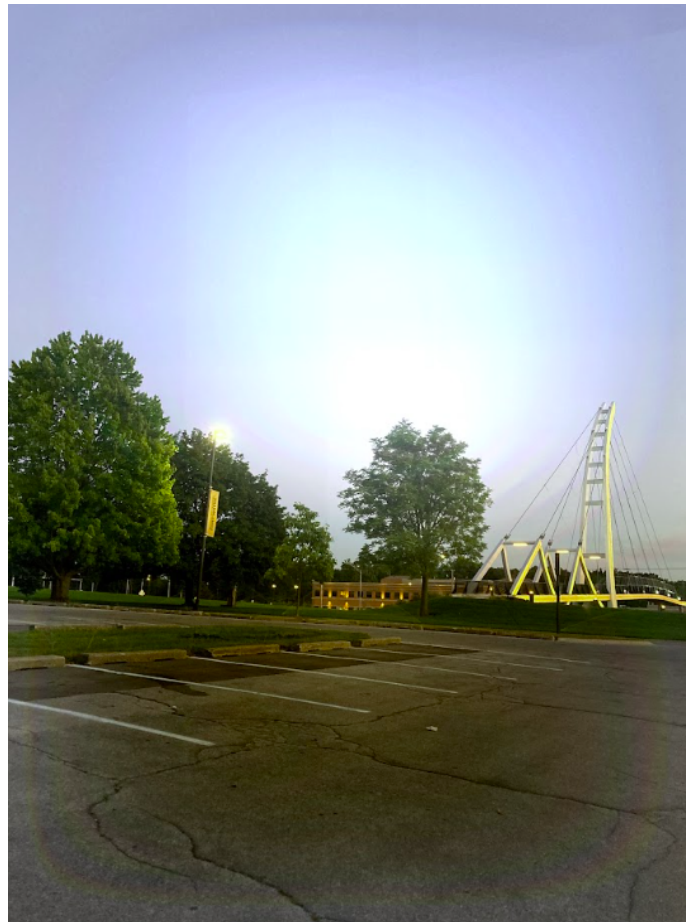


Figure 4.0: Result of “Direct Library function for Gradient Blending” as “panorama_GradLib.jpg”

Model 2 (Direct Image Concatenation)

The Direct Image stitch were very efficient given the quality was untouched but still the image coloration were slightly impacted due to how the pixels were overlapped on one another. Other than the pixel criteria the Panorama image output was pretty good and can be met as a long sustaining solution that might be expected as a proper optimized procedure for image stitching.



Figure 4.1: Result of “Direct Image Concatenation” as “panoramic_image.jpg”.

Model 3 (Gradient Blending Breakdown Technique)

The library function was first followed, whereas the expected output wasn't derived from it. Hence, a break-down of the Gradient Blending method was implemented, with the Laplacian and Poisson Equation respectively in action to calculate the necessary merging. [3]

This Manual step-by-step action helped a lot to derive the ultimate results of a Panorama image, which also had some boderlines. The boderlines stated the image resolution to get lower and image distortion, as the whole procedure were taking as much space as possible of the RAM space in Drive. As a matter of fact it was later downloaded then displayed for the machine itself struggled to keep on running code, while calculating the Poisson Equation for a long time based on how it was occupying what RAM space it wants at different times. Nevertheless, the image result as a Panorama were still considered as a good output structure based on the expected outcome.

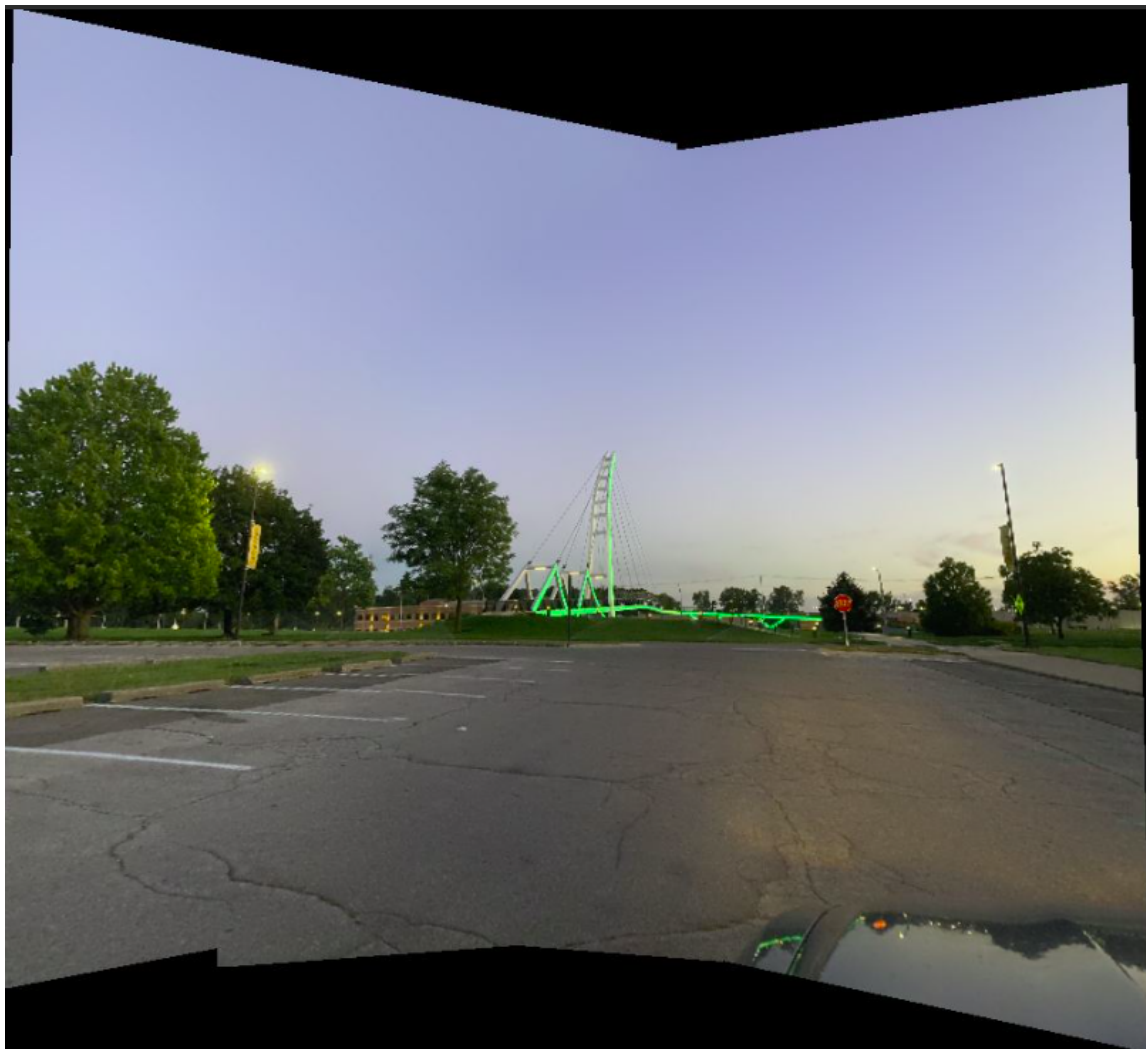


Figure 4.2: Result of “Gradient Blending Breakdown Technique” as “panorama_gradient.jpg”.

Model 4 (Alpha Blending Technique)

While analyzing other blending techniques, the alpha blending mechanism was taken into consideration to be implied for checking a different scope of mechanism. A transparent foreground image is placed on top of a background image through a process called alpha blending.[4]

Even after applying the technique, the blending parameters did not perceive in increasing the image size rather kept image size intact to referencing one image size and overlapping the images which resulted into a mixture of images. The overlap turned into multiple shadows of different pictures merged in together into one frame with no ultimate execution of proper image stitch.

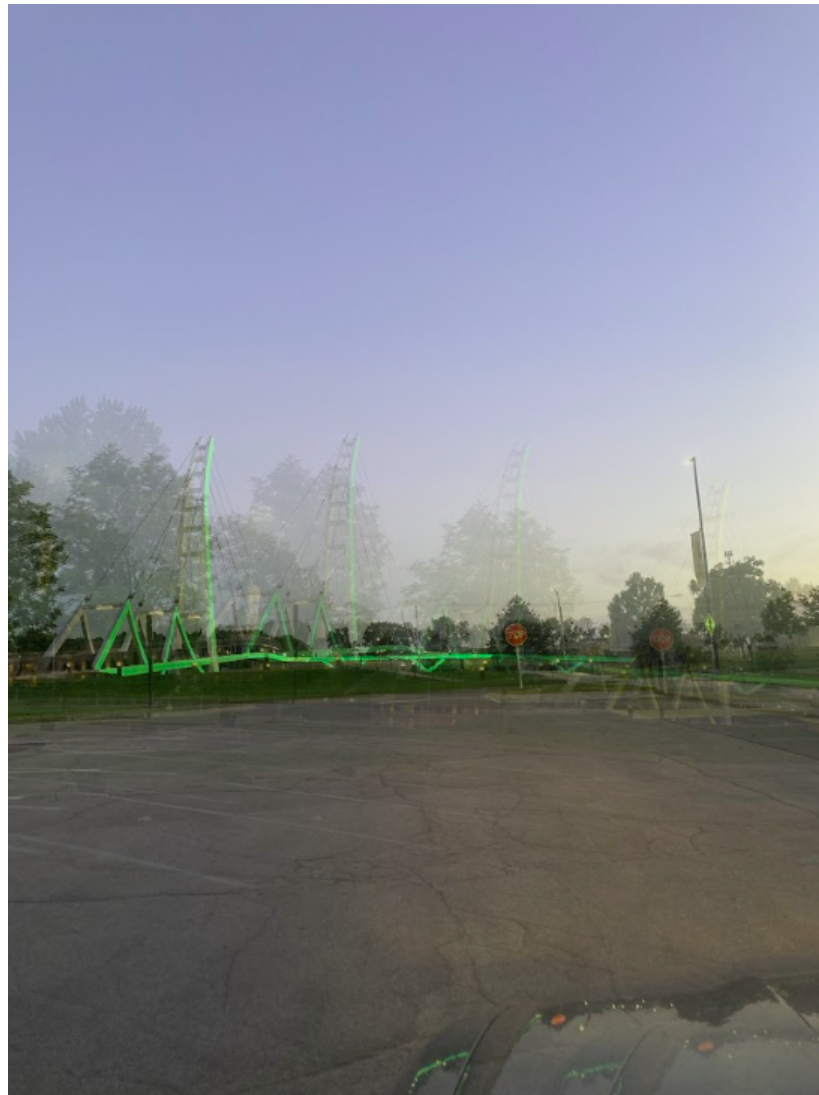


Figure 4.3: Result of “Alpha Blending Technique” as “panorama_alpha.jpg”.

Now, from above it is very evident which procedure gave what sort of optimal outcomes. The results helped to choose out from the several other procedures that were considered before concluding with the given outputs. These outputs helped to make a better conclusion.

Conclusion

While considering all of the above information, two of the above procedures can be suggested as a framework, which may be worked further for building an efficient mechanism for image stitching. As to conclude the procedures, it will be the “Direct Image Concatenation” and “Gradient Blending Breakdown Technique”.

Both techniques were considered for stitching with one having clear resolution as a whole image and the other with perfect blending of the color (pixels) in smooth texture to give it look like a single image of Panorama respectively. This is yet to be worked on more for a combined output as the missing feature of one covers the gap of the other technique. So, with further study these designs are something to be considered in future for a more prominent outcome.

Reference:

1. "Image stitching," ICOI, 03-Dec-2019. [Online]. Available: <https://www.icoi.org/glossary/image-stitching/>. [Accessed: 16-Oct-2023].
2. "OpenCV: ORB (oriented FAST and rotated BRIEF)," Opencv.org. [Online]. Available: https://docs.opencv.org/3.4/d1/d89/tutorial_py_orb.html. [Accessed: 17-Oct-2023].
3. E. Adel, M. Elmogy, and H. Elbakry, "Image stitching system based on ORB feature-based technique and compensation blending," Int. J. Adv. Comput. Sci. Appl., vol. 6, no. 9, 2015. [Accessed: 19-Oct-2023].
4. S. Nayak, "Alpha blending using OpenCV (C++ / python)," LearnOpenCV – Learn OpenCV, PyTorch, Keras, Tensorflow with examples and tutorials, 11-Apr-2017. [Online]. Available: <https://learnopencv.com/alpha-blending-using-opencv-cpp-python/>. [Accessed: 19-Oct-2023].
